

系统理解 Pi

从终端 Coding Agent 到 Agent Harness

核心主线教材 v0.1

源码基线 earendil-works/pi

Commit dcf36c79702ec240b146c45f167ab75ecddd205

读者目标 能运行、追踪、解释并开始修改 Pi；为后续源码深读和 mini Pi 复刻建立框架。

如何使用这本书

这不是一份命令列表，也不是把 Pi 的文件逐行翻译成中文。Pi 是一个正在快速演进的 TypeScript monorepo；若从最大的 UI 文件开始，或者从四千多个 commit 线性阅读，几乎一定会失去方向。本书以一个问题组织：**用户在终端输入一条指令之后，系统怎样把它变成可靠、可恢复、可扩展的 Agent 行为？**

每章遵循相同节奏：

1. 先建立一个可以观察的场景；
2. 再锁定少量源码锚点；
3. 解释数据、状态和控制如何流动；
4. 给出检查问题，确认你不是只“看过”代码。

请不要一口气读完。建议每次只读一章，运行一个命令或一个 faux-provider 测试，然后在自己的笔记中写下：场景、入口、状态所有者、协议、失败路径、仍不理解的问题。

目录

如何使用这本书	ii
第一章 Pi 是什么，以及它不是什么	1
1.1 一句话定位	1
1.2 为什么 Agent 必须是循环	1
1.3 Pi 与 LangChain、LangGraph	2
1.4 本书的完成定义	2
第二章 环境、运行方式与第一条验证锚点	3
2.1 建立可重复环境	3
2.2 三种运行模式	3
2.3 为什么 <code>--list-models</code> 可能没有模型	3
2.4 从 faux provider 开始	3
第三章 从终端输入到 AgentSession	5
3.1 薄入口的价值	5
3.2 <code>main.ts</code> 是产品装配器	5
3.3 为什么需要 AgentSession	5
3.4 <code>cwd</code> 是服务边界	6
第四章 消息、流与状态：Agent 的数据模型	7
4.1 两种消息层	7
4.2 <code>partial</code> 与 <code>final</code>	7
4.3 Agent event 不是日志	7
第五章 Agent loop：模型、工具与下一轮	9
5.1 先看最小伪代码	9
5.2 源码阅读顺序	9
5.3 为什么有两层循环	9
5.4 工具批次的顺序规则	10
5.5 两个必须理解的失败分支	10
第六章 Coding tools：文件、命令与安全边界	11
6.1 工具不只是函数	11
6.2 <code>read</code> 、 <code>edit</code> 、 <code>write</code> 、 <code>bash</code>	11
6.3 截断是上下文工程	11
6.4 Pi 默认不是沙箱	11

第七章 会话、上下文与压缩	12
7.1 transcript 与模型 context 不等同	12
7.2 为什么选择 JSONL	12
7.3 compaction 不是删除	12
第八章 Provider、prompt cache 与 KV cache	13
8.1 Models 与 Provider	13
8.2 faux provider 为什么重要	13
8.3 三个 cache 不要混淆	13
第九章 Extensions、skills 与 TUI	15
9.1 资源为何需要 trust	15
9.2 extension 的边界	15
9.3 TUI 是事件消费者	15
第十章 如何系统学到可以维护 Pi	16
10.1 三条线并行	16
10.2 推荐顺序	16
10.3 能力门槛	16
10.4 第一次读码作业	16
第十一章 事件契约：把异步过程变成可观察事实	18
11.1 为什么事件模型比回调堆更重要	18
11.2 三个时间点	18
11.3 事件序列如何阅读	18
11.4 练习：从事件推状态	19
第十二章 错误、取消、重试：可靠 Agent 的负路径	20
12.1 成功路径只是最小部分	20
12.2 四类失败	20
12.3 abort 不是异常的别名	20
12.4 为什么不让工具返回一段“Error:”文本	20
12.5 retry 的边界	21
12.6 检查问题	21
第十三章 系统提示词、上下文与 Agent 的工作记忆	22
13.1 prompt 不是一个字符串	22
13.2 稳定前缀为何重要	22
13.3 transformContext 的正确位置	22
13.4 上下文窗口是产品约束	22
13.5 练习：画一份请求快照	23
第十四章 测试策略：用确定性模拟代替昂贵的猜测	24
14.1 测试是最短设计文档	24
14.2 四层测试	24
14.3 faux provider 的能力	24
14.4 测试一个工具闭环	24

14.5 读测试的顺序	25
第十五章 用 Git 历史理解设计，而不是淹没在 Git 历史里	26
15.1 何时应该查历史	26
15.2 每个主题四个 commit	26
15.3 安全的 Git 命令	26
15.4 从 history 得到什么	27
第十六章 从 Pi 扩展到更完整的 AI Agent 版图	28
16.1 Pi 已覆盖什么	28
16.2 Pi 不主打什么	28
16.3 一个概念映射	28
第十七章 八周学习路线与学习日志模板	30
17.1 第 1-2 周：核心闭环	30
17.2 第 3-4 周：产品状态	30
17.3 第 5-6 周：长对话与 Provider	30
17.4 第 7-8 周：扩展与交互	30
17.5 每次学习日志	30
第十八章 源码地图：从问题定位到正确文件	32
18.1 不要用“全仓搜索”代替架构	32
18.2 问题到包的第一映射	32
18.3 核心文件的阅读顺序	32
18.4 一个实际定位例子	33
18.5 命令工具箱	33
第十九章 Mini Pi 实验室：用复刻证明你真的理解	34
19.1 实验室的角色	34
19.2 建议的最小数据模型	34
19.3 阶段一：scripted model	34
19.4 阶段二：工具和错误	35
19.5 阶段三：并发与稳定历史	35
19.6 阶段四：session 与 compaction	35
19.7 mini Pi 的验收表	35
第二十章 Provider adapter 解剖：把不同模型协议收敛为一个流	36
20.1 为什么 adapter 是必要复杂度	36
20.2 adapter 的输入与输出	36
20.3 先读 faux，再读一个真实 adapter	36
20.4 认证不应泄漏到 Agent loop	36
20.5 Adapter 阅读问题	37
第二十一章 终端 UI 深读前需要懂的渲染机制	38
21.1 为什么终端 UI 仍是一个独立系统	38
21.2 差量渲染的抽象	38
21.3 Pi 的两层 UI	38

21.4 输入的正确方向	38
21.5 第一次 TUI 作业	39
第二十二章 术语表、检查清单与下一次阅读的人口	40
22.1 核心术语表	40
22.2 第一次完整调用链的空白模板	41
22.3 阅读 Agent loop 前的自测	41
22.4 阅读 Provider 前的自测	41
22.5 阅读 Coding Agent 前的自测	42
22.6 建议的学习记录评分法	42
22.7 下一次打开源码时，从这里开始	42
第二十三章 第一天的 30 分钟任务	43
23.1 目标	43
23.2 时间盒	43
23.3 今天不要做什么	43
下一版会补什么	44

Pi 是什么，以及它不是什么

Pi 是一个 **Agent harness**：它为语言模型提供消息协议、流式响应、工具调用、会话、上下文管理、多 Provider 适配和终端交互外壳。其最知名的产品是终端 Coding Agent，但仓库并不等于“一个调用模型的 CLI”。

包	职责	不应该负责什么
packages/ai	Provider、模型目录、认证、协议适配、流	文件编辑、终端 UI
packages/agent	Agent state、turn、tool loop、事件	具体 coding tool、主题
packages/coding-agent	CLI、工具、session、扩展、产品策略	重写 Agent loop
packages/tui	输入编辑、组件、ANSI 差量渲染	模型调用、会话策略

Pi 的关键不是某一个 prompt，而是边界。模型层不知道文件系统；Agent core 不知道主题和键盘；UI 消费事件而不拥有第二份对话状态。只要边界保持，任何一层都可以替换。

```

user -> LLM -> tool call -> host executes read/bash/edit
                    ^                               |
                    |----- tool result -----+

```

模型不会直接读磁盘。它只能生成“调用 read，参数为 path”的结构化意图；Pi 负责验证、执行、记录结果，再把结果作为消息送回模型。一次用户请求可以包含多个 turn：每个 turn 有一次模型响应，可能跟着一批工具执行。

1.3 Pi 与 LangChain、LangGraph

Pi 当前仓库没有 LangChain 依赖。它和 LangChain/LangGraph 都处理模型、tools、memory、callback，但抽象中心不同：Pi 把单 **Agent 的消息-工具循环** 作为核心原语；LangGraph 更适合将业务显式建模为多个节点和边的状态图。Pi 适合学习交互式 coding agent 的运行；若将来要做多 Agent 审批流、复杂 workflow 编排，再补 LangGraph 更合理。

1.4 本书的完成定义

读完核心版，你应该能：

- 解释四层包如何协作；
- 追踪一次 prompt 从 CLI 到 tool result 再到最终回答；
- 运行无需 API key 的 faux-provider 测试；
- 知道 session、context、Provider cache 与模型内部 KV cache 的边界；
- 选择一个低风险行为做出最小改动，并找到对应测试。

第 2 章

环境、运行方式与第一条验证锚点

2.1 建立可重复环境

Pi 当前要求 Node 22.19 或更高版本。请在完整 clone 根目录执行：

```
cd D:\projects\pi-source-analysis
npm install --ignore-scripts
.\pi-test.ps1 --help
.\pi-test.ps1 --list-models
```

`pi-test.ps1` 是 Windows 的源码启动包装：它定位根目录 `tsx.cmd`，然后执行 `packages/coding-agent/src/cli.ts`。它可附加 `--no-env`，清除常见 Provider 凭据，避免实验意外使用真实账号。

2.2 三种运行模式

在 `packages/coding-agent/src/main.ts` 中，`resolveAppMode(parsed, stdinIsTTY, stdoutIsTTY)` 的规则非常重要：

```
--mode rpc           => rpc
--mode json           => json
--print 或任一流非 TTY => print
其他                  => interactive
```

这证明终端 UI 不是 Agent 的前提。相同的 session/runtime 可以被人类终端、管道脚本和 RPC 客户端复用。

2.3 为什么 `--list-models` 可能没有模型

`--list-models` 并非简单列出一个硬编码数组。它经过模型注册表和认证可用性判断。没有 API key 或 OAuth 账号时，Pi 正确地提示 “No models available” 并指向 `provider/model` 文档。它不是安装失败，而是模型层刻意不把 “已知模型” 误报为 “当前可请求模型”。

2.4 从 faux provider 开始

不要把第一次端到端学习建立在真实 API 上。Pi 自带 faux provider，可以确定性地输出指定的 assistant message、thinking、tool call、延迟与 usage。

```
cd packages\agent
.\node_modules\.bin\vitest.cmd --run test\e2e.test.ts `
  -t "executes tools and tracks pending tool calls"
```

该测试位于 `packages/agent/test/e2e.test.ts`。它让 `faux` 模型先返回一个 `calculate tool call`，再返回包含计算结果的文本；测试断言：

- `transcript` 中存在 `tool result`；
- 最后消息是 `assistant` 且含正确答案；
- `pendingToolCalls` 在 `tool_execution_start` 时包含 `call ID`，在 `end` 时空。

常见误解

不要把“测试通过”理解为“理解完成”。它只是你的第一个固定观测点：以后读 `loop`、加工具、研究并发时都可以回到这条测试验证基本闭环没有被破坏。

第 3 章

从终端输入到 AgentSession

3.1 薄入口的价值

`packages/coding-agent/src/cli.ts` 很短：设定进程名，标记 `PI_CODING_AGENT`，配置 HTTP dispatcher，然后调用 `main(process.argv.slice(2))`。这种薄入口避免 import 副作用、参数解析、网络配置、业务状态和 UI 互相纠缠。

3.2 `main.ts` 是产品装配器

`main.ts` 不是 Agent loop。它依次处理参数、stdin/附件、session 选择、settings、project trust、resource loader、认证和模型，然后创建 runtime 并分派 mode。你读它时只回答四个问题：

1. 什么条件决定 mode？
2. 哪个对象负责当前 cwd？
3. session 从何处恢复或创建？
4. Agent、工具、模型和 extensions 在哪里装配？

3.3 为什么需要 AgentSession

`packages/agent/src/agent.ts` 的 Agent 是通用 runtime：它维护消息、工具、streaming 状态和队列。Coding Agent 还需产品能力：JSONL session、模型选择、compaction、bash、project resources、extension lifecycle。于是 `packages/coding-agent/src/core/agent-session.ts` 包装 core Agent，提供 coding-agent 专有策略。

```
CLI / print / RPC / interactive
  |
  AgentSessionRuntime
    |
    AgentSession
      |
      Agent
        |
        pi-ai Model / Provider
```

3.4 cwd 是服务边界

`agent-session-runtime.ts` 在 session 切换时不会只修改一个 `cwd` 字符串。它会关闭旧 session 生命周期、让 host 解除旧 UI/extension 绑定、按新 `cwd` 重建 services，然后绑定新 session。原因是 settings、skills、extensions、context files 和 trust 都可能依赖项目目录。复用旧项目资源会构成权限和一致性问题。

第 4 章

消息、流与状态：Agent 的数据模型

4.1 两种消息层

Pi 区分 **AgentMessage** 与 LLM 可理解的 **Message**。前者可以包含应用专用记录；后者只能是 Provider 支持的 user、assistant、toolResult 等标准消息。边界是：

```
AgentMessage[]
-> transformContext() // 裁剪、注入、压缩
-> convertToLlm()      // 过滤或转换自定义消息
-> Message[]
-> Provider
```

这条边界避免 UI 通知、分支摘要或内部 session entry 意外泄漏给模型，同时允许产品在调用模型前重写上下文。

4.2 partial 与 final

Provider stream 可能产生 start、text delta、thinking delta、tool-call delta、done/error。用户希望马上看到 token；session 却只能保存完整、稳定的消息。因此 `packages/agent/src/agent-loop.ts` 的 `streamAssistantResponse()` 在 start 时持有 partial assistant，在每个 delta 更新它，最终通过 `response.result()` 获取 final assistant message。

核心结论

partial message 是渲染状态，不是持久事实。若把 partial 直接写入 JSONL 或作为下一轮完整上下文，工具参数和 usage 都可能不完整。

4.3 Agent event 不是日志

典型事件序列：

```
agent_start
turn_start
message_start(user) / message_end(user)
message_start(assistant)
message_update(...)
message_end(assistant)
```

```
tool_execution_start / update / end  
message_start(toolResult) / message_end(toolResult)  
turn_end  
agent_end
```

`Agent.processEvents()` 的顺序是：先归约内部 state，后按注册顺序 await listener。这个顺序让 session persistence、UI 和 tool hook 观察同一份已更新的状态，而不是竞争谁先看到 assistant message。

第 5 章

Agent loop: 模型、工具与下一轮

5.1 先看最小伪代码

```
append user prompt
while true:
    assistant = stream model(context)
    append final assistant
    calls = assistant.toolCalls
    results = execute calls
    append results
    if no calls: stop
```

真实 Pi 在此基础上增加 context transform、abort、错误处理、parallel/sequential batch、steering、follow-up、hooks、dynamic next-turn state 和 compaction 协作。

5.2 源码阅读顺序

打开 `packages/agent/src/agent-loop.ts`，不要从底部 helper 开始。依次读：

1. `runAgentLoop()`：新 prompt 如何进入 context；
2. `runLoop()`：双层循环如何组织；
3. `streamAssistantResponse()`：唯一的模型调用边界；
4. `executeToolCalls()`：如何选择串行或并行；
5. `tool prepare/execute/finalize`：验证、hook、执行、错误和结果。

5.3 为什么有两层循环

内层处理“还有 tool call”或“有 steering message”；外层在本应结束时查询 follow-up queue。语义不同：

- **abort**：取消正在进行的流或工具；
- **steering**：当前 tool batch 完成后优先插入的新指令；
- **follow-up**：Agent 原本完成后才开始的新工作。

把三者都实现为“追加 prompt”会导致 tool 未完成、历史顺序混乱或用户意图被错误解释。

5.4 工具批次的顺序规则

Pi 默认可并行执行工具，但 `preflight` 按顺序完成；若任意工具要求 `executionMode: "sequential"`，整批改为串行。并发时 UI 的 `tool_execution_end` 可以按完成顺序即时出现；而持久化 `tool result` 仍按 `assistant` 原始 `source order` 写入 `transcript`。

这两个顺序同时存在是成熟实现常见的细节：人想尽快看到完成；模型和 `session` 需要稳定、可重放的历史。

5.5 两个必须理解的失败分支

1. `assistant` 因输出长度停止且带 `tool call`：参数可能截断，即使 JSON 恰好可解析也不安全；Pi 拒绝执行并要求模型重发。
2. 工具抛异常：Agent 转成 `isError: true` 的 `tool result`，让模型有机会修复调用；不能静默吞错，也不应让整个 `session` 崩溃。

第 6 章

Coding tools: 文件、命令与安全边界

6.1 工具不只是函数

一个 Coding Agent tool 至少有模型可读描述、参数 schema、执行器、可展示结果、details、错误语义、取消支持和输出边界。Coding Agent 的内建工具在 `packages/coding-agent/src/core/tools/`；装配入口在 `core/tools/index.ts`。

6.2 read、edit、write、bash

工具	主要目的	主要风险
read	获取文件上下文	大文件、二进制、路径出界、token 爆炸
edit	受控 find/replace	旧内容不匹配、重复替换、并发修改
write	创建/覆盖内容	误覆盖、目录边界、部分写入
bash	执行命令	shell 注入、挂起、子进程、凭据和网络

6.3 截断是上下文工程

模型上下文有限。一次巨大命令输出或读取依赖目录，可能挤掉用户任务和系统提示词。Pi 在工具返回层截断，而非等 Provider 报 context overflow；并把截断事实明确放入返回内容。这样模型、UI、session 对看到的结果一致。

6.4 Pi 默认不是沙箱

Pi README 明确说明：它以启动进程的用户权限访问文件、进程、网络 and 凭据，没有内建权限系统。Prompt 不是权限系统；让模型“不要删除文件”不能限制 shell。需要更强边界时，使用容器、VM、OpenShell 或 Gondolin 等外部方案。学习工具实现时，请始终把“路径检查”、“输出限制”与“真正隔离”视为三个不同层次。

第 7 章

会话、上下文与压缩

7.1 transcript 与模型 context 不等同

Pi 可以在 session 文件中保存 user/assistant/tool result 之外的分支、摘要、custom entry；模型每轮看到的却是经过 transform 和 conversion 的上下文。这样做的目的不是隐藏历史，而是让可恢复记录、UI 呈现和有限 context window 可以分别优化。

7.2 为什么选择 JSONL

`packages/coding-agent/src/core/session-manager.ts` 采用逐行 JSON：

- 新 entry 可追加，不必重写完整对话；
- 进程中断后，已完成行仍可读取；
- 导出、分支和诊断能以 entry 为单位处理；
- header、summary、custom entry 可自然共存。

恢复时不应静默跳过坏行；若历史被截断，系统必须把错误位置和恢复策略明确给用户。

7.3 compaction 不是删除

当上下文接近模型窗口，Pi 的 compaction 选择一段历史生成 summary，并以显式 entry 取代模型上下文中的旧细节。正确的 compaction 要满足：

1. 用户能知道摘要发生过；
2. summary 有清晰切点和来源；
3. 失败或 abort 不会损坏原 session；
4. 后续消息、queue 和 retry 的顺序保持一致。

`packages/coding-agent/src/core/compaction/` 是后续深读专题；本阶段先牢牢记住：“保存了什么”、“显示了什么”、“发给模型什么”可能不同，且差异必须可解释。

第 8 章

Provider、prompt cache 与 KV cache

8.1 Models 与 Provider

`packages/ai/src/models.ts` 把职责拆为：

- **Provider**: 自己的 model list、认证语义、请求流；
- **Models**: 查找 provider/model、解析认证、合并请求选项、委托 stream；
- **adapter**: 把 OpenAI、Anthropic、Google 等不同协议归一为 Pi message/event。

Agent loop 不应该出现 “if provider === anthropic”。若新增 Provider 必须修改 loop，抽象边界就失败了。

8.2 faux provider 为什么重要

`packages/ai/src/providers/faux.ts` 不是玩具。它让测试精确配置模型响应、工具调用、token 切片、延迟、usage 和 prompt cache 模拟。真实 Provider 测试会受到密钥、账单、网络、模型升级和限流影响；核心 Agent 不变量必须优先由 faux provider 守护。

8.3 三个 cache 不要混淆

概念	所在位置	Pi 做什么
模型 KV cache	GPU/推理服务器内部 attention tensor	Pi 不实现、不存储、不调度
Provider prompt cache	OpenAI/Anthropic 等服务端缓存稳定 prompt 前缀	Pi 设置 session/cache 相关协议字段，统计 cache usage
本地 session/context	Pi JSONL、summary、session ID	Pi 管理恢复与稳定上下文，不是 tensor cache

在 `packages/agent/src/agent.ts`, `sessionId` 被传给每轮流。不同 adapter 再将其映射为合适机制：OpenAI Responses 可使用 `prompt_cache_key`；Anthropic 可插入 `cache_control` marker；部分后端使用 session affinity header。相关类型在 `packages/ai/src/types.ts`, OpenAI key 长度处理在 `api/openai-prompt-cache.ts`。

常见误解

prompt cache 命中不等于 Pi 在本地保存了 KV tensor；它只表示后端可能复用稳定前缀。真正影响命中的工程行为包括保持 system prompt、工具定义和早期消息的稳定顺序，以及避免无意义地重写 session ID。

第 9 章

Extensions、skills 与 TUI

9.1 资源为何需要 trust

项目目录可以携带 AGENTS.md、skills、prompt templates、themes、extension。它们可能改变 Agent 行为，extension 甚至能运行代码。因此 Pi 先做 project trust，再加载项目资源；不能先执行本地 extension 后再询问用户是否信任。

9.2 extension 的边界

extension 不应直接抓取 AgentSession 私有字段。Pi 通过 resource loader、extension runner、事件/hook/context 暴露能力。这样 lifecycle 可以记录和清理，错误可以诊断，session 切换时也能正确 teardown。

9.3 TUI 是事件消费者

packages/tui 是独立包，提供 terminal、editor、component、keybinding 和差量渲染。Coding Agent 的 modes/interactive/interactive-mode.ts 负责把 session event 映射为组件状态。

正确方向：

```
keyboard -> AgentSession.prompt / abort / steer  
AgentSession event -> component state -> differential render
```

错误方向是让 UI 自行 append assistant 文本、执行工具或维护第二份 transcript；那会让 print、RPC、session restore 和 interactive 产生不同事实。

第 10 章

如何系统学到可以维护 Pi

10.1 三条线并行

- 60%: 当前代码纵向切片, 理解今天怎样工作;
- 25%: mini Pi 复刻, 证明能重新实现不变量;
- 15%: 按需 Git 考古, 解释设计为何变成今天这样。

10.2 推荐顺序

1. faux provider + 一轮文本流;
2. 单工具 loop;
3. Agent event/state 与 abort;
4. read tool 与输出截断;
5. AgentSession 和 JSONL session;
6. compaction;
7. bash 与安全;
8. Models/auth/一个真实 Provider;
9. extension/skill/trust;
10. pi-tui 与 interactive mode;
11. 每主题回溯四个关键 commit: 引入、重构、bug fix、当前。

10.3 能力门槛

门槛	证明
最小闭环	能画 tool call 的 event/transcript 顺序并运行 faux test
安全修改	能为已有行为补测试或小修复, 说明影响边界
产品重建	能从零实现 session、工具、context 与 print CLI
成熟设计解释	能用当前代码和四个 commit 解释 compaction/OAuth/TUI 等专题
贡献能力	能定位 issue、提出最小 patch、添加确定性 regression test

10.4 第一次读码作业

1. 运行第 2 章的 faux-provider test;

2. 在 `packages/agent/test/e2e.test.ts` 找到该测试的两条 faux response;
3. 追 `Agent.prompt()`、`runLoop()`、`executeToolCalls()`;
4. 画出 user、assistant(tool call)、toolResult、assistant(final) 四条 transcript entry;
5. 写下: tool result 为什么要有 call ID? 为什么 final assistant 不是第一次 assistant?

第 11 章

事件契约：把异步过程变成可观察事实

11.1 为什么事件模型比回调堆更重要

一个 Agent 同时会发生很多事情：模型在输出文字；模型可能生成 tool call；工具正在运行；用户可能要求取消；UI 要及时显示；session 要可靠写盘。若每个模块随意直接调用另一个模块，就会产生隐形时间依赖：UI 先看到结果还是 session 先落盘？tool hook 在 assistant message 入历史前还是后执行？取消发生时哪个 promise 负责收尾？

Pi 的答案是将重要阶段建模为 **AgentEvent**。事件不是调试日志，而是跨层协议。它使每个消费者只订阅自己关心的事实：

- session listener 在 `message_end` 写入最终消息；
- TUI 在 `message_update` 更新流式组件；
- footer 在 usage 或 turn 结束时更新 token/cost；
- extension hook 在规定时点检查工具调用；
- 测试把事件序列作为行为规范。

11.2 三个时间点

理解 event 时，不要只问“发生了什么”，还要问“此刻 state 中有什么”。以 tool call 为例：

1. `message_end(assistant)`：assistant 的完整 tool call 已进入 transcript；
2. `tool_execution_start`：参数已经解析并通过基本验证，`pendingToolCalls` 包含 call ID；
3. `tool_execution_end`：工具已成功或失败，`pending` 集合已移除该 ID；紧随其后才会有稳定的 tool result message。

这也是为什么 `Agent.processEvents()` 先更新 state 再 await listener。若 listener 先执行，UI、session 和 extension 会观察到旧状态，出现难以复现的竞态。

11.3 事件序列如何阅读

把 `packages/agent/test/e2e.test.ts` 当作协议说明。它不只检查最终答案，也检查工具执行前后 `pendingToolCalls` 的集合。一个更完整的人工追踪表如下：

事件	transcript 状态	典型消费者
<code>agent_start</code>	未必已有新增消息	loading indicator、run timer
<code>message_start(user)</code>	用户消息正在进入 run	echo/输入确认

<code>message_start(assistant)</code>	partial assistant 已建立	assistant component
<code>message_update</code>	partial 内容递增	streaming markdown/thinking
<code>message_end(assistant)</code>	final assistant 可持久化	session、tool preflight
<code>tool_execution_start</code>	call 标记为 pending	tool card、取消状态
<code>tool_execution_update</code>	中间进度可显示	bash progress、长任务 UI
<code>message_end(toolResult)</code>	结果可成为下一轮 context	session、transcript view
<code>turn_end</code>	一个模型响应及其工具批次完成	compaction/retry 决策
<code>agent_end</code>	本轮不再发事件	final flush、idle UI

11.4 练习：从事件推状态

假设你看到以下片段：

```
message_end(assistant with toolCall "edit")
tool_execution_start(edit, call-7)
tool_execution_end(edit, call-7, isError=true)
message_end(toolResult call-7)
turn_end
turn_start
```

请推断：第一轮是否已经结束？模型是否已经收到失败信息？为什么还有第二个 `turn_start`？正确答案是：工具失败不是 run 失败；失败被标准化为 tool result，下一轮模型获得修正自己参数或策略的机会。

第 12 章

错误、取消、重试：可靠 Agent 的负路径

12.1 成功路径只是最小部分

演示 Agent 时常只展示“read 文件后回答”。真实使用中更常见的是网络断开、Provider 过载、无效工具参数、文件不存在、shell 退出非零、用户突然取消、context overflow。一个成熟 harness 的质量，更多由失败语义而不是 happy path 决定。

12.2 四类失败

失败来源	例子	正确归宿
输入/配置	没有模型、无凭据、坏 session ID	启动诊断或明确 CLI error
Provider 流	timeout、5xx、rate limit、stream error	final assistant error 或受控 auto-retry
工具执行	文件不存在、schema 不符、bash 失败	<code>isError=true</code> tool result
用户控制	abort、session 切换、关闭终端	AbortSignal, 最终状态为 aborted

12.3 abort 不是异常的别名

`Agent.abort()` 触发当前 run 的 `AbortController`。该 signal 被交给 Provider stream 和 tool execute。abort 的语义是“用户不再需要这项工作”；它并不等同“系统出错”。因此 final assistant 的 `stopReason` 可以是 `aborted`，UI 可以显示已取消而不是红色错误，session 仍可以保留已经完成的历史。

对 bash 特别要警惕：取消上层 promise 不保证子进程已经停止。Coding Agent 为命令执行维护更具体的 process 管理逻辑；这是 product tool 层的职责，不应塞进通用 Agent loop。

12.4 为什么不让工具返回一段“Error:”文本

工具应在失败时抛出异常，由 Agent 统一转成 error tool result。好处是：

- 有单独的 `isError` 机器可读位，而不是依赖字符串前缀；
- UI 可以以错误样式展示；

- hook/metrics 能区分成功与失败；
- 模型收到完整、稳定的失败消息而非半成功内容。

12.5 retry 的边界

自动重试只适用于暂时性 Provider 失败，例如 rate limit 或 5xx；它不应重试错误 tool 参数、权限拒绝或用户 abort。更重要的是重试不得重复执行有副作用的工具。Pi 将 Provider retry、tool execution、session compaction 分别处理，正是为了避免一个笼统“catch then retry”带来的重复写文件和重复命令。

12.6 检查问题

1. 如果文本已经 stream 到一半，用户按取消，session 中应保存什么？
2. 如果 read 工具失败，为什么下一轮仍值得调用模型？
3. 为什么 provider retry 不应该包住“模型调用 + 工具执行”的整个 loop？

第 13 章

系统提示词、上下文与 Agent 的工作记忆

13.1 prompt 不是一个字符串

初学者常把系统提示词理解为一段固定文本。对 Coding Agent 而言，模型每轮请求的有效上下文至少包含：

- system prompt：角色、工具约束、行为规则；
- 对话历史：用户、assistant、tool result；
- 工具定义：名称、description、JSON schema；
- 动态资源：项目 instruction、skills、prompt template；
- Provider 选项：thinking、cache、session affinity、transport。

它们共同决定模型能看到什么、能调用什么、后续请求能否复用缓存。

13.2 稳定前缀为何重要

很多 Provider 的 prompt cache 依赖稳定前缀。若每一轮随机重排 system prompt、工具定义或早期 context，后端很难复用已处理部分；成本、延迟和 cache read 都会变差。Pi 的 session ID、cache retention 和 context 构造因此不是微小性能优化，而是 Agent 长对话体验的一部分。

13.3 transformContext 的正确位置

`transformContext` 在 `AgentMessage` 层工作，发生在转换为 LLM Message 之前。它适合：

1. 裁剪过长工具输出；
2. 注入显式 summary；
3. 筛掉仅供 UI 使用的 entry；
4. 加入外部但可信的项目上下文。

它不应该偷偷删掉用户历史而不留下可审计痕迹。正确做法是 compaction：将旧片段转为 summary entry，并令用户、session 和模型都能解释这个转换。

13.4 上下文窗口是产品约束

模型的 context window 并不是越大越不用管理。长上下文仍有成本、延迟、注意力稀释和 Provider 限制；工具输出会快速放大 transcript。Pi 的 compaction、truncate、token estimate、cache statistics 都是在处理这个产品约束。学习时把“上下文预算”当作和内存预算同样真实的资源。

13.5 练习：画一份请求快照

任选一条测试，在模型调用点写下：

```
systemPrompt:
messages:
tools:
model:
sessionId:
thinking level:
cache options:
```

下一轮在 `append tool result` 后再次写一份。比较两份快照，便能看清 Agent loop 为什么不是普通聊天。

第 14 章

测试策略：用确定性模拟代替昂贵的猜测

14.1 测试是最短设计文档

Pi 的测试价值不在覆盖率数字，而在把异步行为固定为可读的契约。比起从 production code 猜“并行工具完成后结果如何排序”，先看 `packages/agent/test/agent-loop.test.ts` 的断言会更快得到答案。

14.2 四层测试

1. **协议单测**：schema、消息转换、cache option、路径/截断 helper。
 2. **loop 单测**：给定 scripted assistant message，断言 event 与 transcript。
 3. **session/product 测试**：JSONL、恢复、fork、compaction、settings、tools。
 4. **受控交互测试**：TUI、CLI/RPC mode；少量真实 Provider smoke test。
- 越靠内层，越应确定、快速、无需凭据。越靠外层，越少而精。

14.3 faux provider 的能力

`providers/faux.ts` 可设置一串 response，或提供根据 context 返回 message 的 factory。它可以模拟：

- text/thinking/tool-call content blocks；
- 每秒 token 数和 token chunk 大小；
- usage 与 cache read/write；
- 第二轮是否正确看到第一轮 tool result；
- abort 时是否产生正确 final message。

这让你能写一种真正有价值的测试：“给定 assistant 先要求 calculate，再给最终文本，Agent 应生成哪些状态与事件？”而不是“请真实模型算 2+2，希望它合作”。

14.4 测试一个工具闭环

理想的断言至少包括：

```
assistant message has toolCall(call-1)
tool_execution_start(call-1)
toolResult.toolCallId === call-1
toolResult.isError === false
```

```
next assistant message sees result  
pendingToolCalls is empty after completion
```

若只断言最终自然语言答案，即使 tool 根本没执行、模型碰巧猜对了，也会测试通过。

14.5 读测试的顺序

1. 先读 test name：作者想保证什么？
2. 再读 setup：faux model 和 tools 被如何配置？
3. 列出断言的事件、state 和消息；
4. 最后跳到实现中找使断言成立的最短函数路径。

这种 “test -> implementation” 路径比先读几千行 runtime 更适合初学复杂 Agent。

第 15 章

用 Git 历史理解设计，而不是淹没在 Git 历史里

15.1 何时应该查历史

当前 Pi 有数千个 commit。线性阅读会混入版本 bump、模型目录生成、格式化和 CI 维护，学习效率很低。应只在以下问题出现时查 history：

- 这个看似复杂的分层为什么存在？
- 某个兼容字段或额外 event 在防哪类 bug？
- 为什么这里选择并发/串行、summary/删除、session ID/随机 ID？
- 哪个测试是回归测试，它最初修了什么？

15.2 每个主题四个 commit

对一个文件或概念，最多选择：

1. 初次引入：需求最原始的形状；
2. 关键重构：职责为何重新划分；
3. 高价值 bug fix：不变量曾如何被打破；
4. 当前版本：最终形态。

例如 tool execution 可以围绕 `agent-loop.ts` 查并行批次、完成顺序、truncated tool call 拒绝、per-tool sequential override。每次都比较父提交和目标提交，先写“变化前什么会失败”，再读 diff。

15.3 安全的 Git 命令

```
git log --follow -- packages/agent/src/agent-loop.ts
git log --follow -- packages/coding-agent/src/core/agent-session.ts
git show --stat <commit>
git diff <commit>^ <commit> -- <path>
git show <commit>:<path>
```

不要为了阅读历史频繁把主工作目录切到旧 commit。使用 `git show`、临时 worktree 或只读 detached checkout；实验代码放独立目录。

15.4 从 history 得到什么

历史不是为了记 SHA，而是为了捕获设计动机。例如“state before listener”这种看似微小的顺序，往往来自实际 race bug；它比注释更能说明为什么不能随意重排。把这种结论写入你的 invariants 笔记，才是考古的产物。

第 16 章

从 Pi 扩展到更完整的 AI Agent 版图

16.1 Pi 已覆盖什么

Pi 是学习单 Agent engineering 的高密度案例：

- 流式 LLM 协议统一；
- structured tool calling；
- context/token/cache usage；
- session、compaction、retry；
- 多 Provider、认证和 OAuth；
- coding tool 安全、containerization；
- event-driven terminal UX。

16.2 Pi 不主打什么

以下不是 Pi 的主要实现重心：

- 多 Agent 协作、planner/worker/supervisor 图；
- RAG、向量数据库、retrieval ranking；
- 模型训练、推理引擎、GPU 显存/KV cache 管理；
- 企业级长期记忆、评估平台和 observability backend。

这不是缺陷，而是边界清晰。理解 Pi 后，补 LangGraph 学图式编排，补 RAG 学检索，补 vLLM/TGI 学推理服务，补 eval/trace 系统学可靠性，路径会更自然。

16.3 一个概念映射

问题	Pi 的答案	可后续学习的相邻领域
单 Agent 如何反复调用工具？	Agent loop + tool result	ReAct、function calling
多个步骤如何显式分叉？	有限的 queue/session 分支	LangGraph/workflow engines
如何让模型知道私有资料？	project context/skills，不是检索系统	RAG/vector search
如何降低重复长 prompt 成本？	Provider cache/session ID	inference cache/KV cache

如何隔离危险命令?	外 部	sand-	policy	sandbox/VM	orches-
	box/containerization		tration		

第 17 章

八周学习路线与学习日志模板

17.1 第 1–2 周：核心闭环

- 跑 help、model list、faux provider e2e;
- 读 Agent.prompt、runLoop、streamAssistantResponse;
- 画 user -> assistant(tool) -> toolResult -> assistant 的四消息图;
- 写 mini loop, 只支持一个 echo/read tool。

17.2 第 3–4 周：产品状态

- 读 AgentSession、session manager、read tool;
- 实现 JSONL append/reload 和路径受限 read;
- 学 abort、error tool result、parallel/sequential;
- 为 mini loop 增加 faux tests。

17.3 第 5–6 周：长对话与 Provider

- 读 compaction、token estimate、cache usage;
- 选一个 OpenAI 或 Anthropic adapter, 不同时读多个;
- 区分 prompt cache 和 KV cache;
- 实现显式 summary, 不静默删历史。

17.4 第 7–8 周：扩展与交互

- 读 resource loader、trust、extension runner;
- 独立读 pi-tui 的 component/render contract;
- 最后进入 interactive mode;
- 选择一个主题回溯四个关键 commit, 写设计说明。

17.5 每次学习日志

场景:

运行命令与可观察输出:

测试名称:

入口 -> 核心 -> 输出:

状态由谁拥有:

消息/事件不变量:

失败、取消、重试路径:

一个相关 `commit` 与它解决的问题:

我的最小实验:

仍不理解的问题:

当这些日志积累到十几篇，你将拥有比一份泛泛教程更有价值的个人 Pi 设计文档。

第 18 章

源码地图：从问题定位到正确文件

18.1 不要用“全仓搜索”代替架构

Pi 的当前实现中，同一个词可能出现在 Provider、Agent、Coding Agent、TUI 和测试多个层。搜索“session”会产生大量结果，却不告诉你哪一层拥有 session；搜索“tool”更会混合 schema、执行、渲染、extension 和文档。先按问题选择包，再用符号和测试缩小范围。

18.2 问题到包的第一映射

你正在问什么？	第一站	第二站
命令行 flag 为何如此处理？	<code>coding-agent/src/cli/args.main.ts</code>	
为什么进入 <code>print/RPC/interactive</code> ？	<code>main.ts</code>	<code>modes/</code>
模型为何收到某些消息？	<code>agent-loop.ts</code>	<code>types.ts/context transform</code>
工具为何执行/被拒绝/并发？	<code>agent-loop.ts</code>	<code>coding-agent/core/tools/</code>
状态何时写入 session？	<code>agent-session.ts</code>	<code>session-manager.ts</code>
某 Provider 请求长什么样？	<code>ai/src/api/<adapter>.ts</code>	<code>models.ts/auth</code>
为什么 cache usage 变化？	<code>ai/src/types.ts</code>	<code>Provider adapter/faux test</code>
某个终端组件为何刷新？	<code>tui/src/</code>	<code>interactive-mode.ts</code>
extension 能做什么？	<code>core/extensions/</code>	<code>resource loader/trust manager</code>

18.3 核心文件的阅读顺序

1. `packages/agent/src/types.ts`: 先定义词汇。不要猜 `AgentTool`、`AgentEvent`、`AgentContext` 的字段。
2. `packages/agent/src/agent.ts`: 看 state、public API、listener 屏障、`AbortController`、`queues`。
3. `packages/agent/src/agent-loop.ts`: 看每一轮的实际控制流。
4. `packages/ai/src/types.ts` 与 `models.ts`: 看模型流、Provider、auth 的契约。
5. `packages/coding-agent/src/core/agent-session.ts`: 看产品策略如何附着在 core 上。
6. `packages/coding-agent/src/core/tools/`: 看真实 side effect 如何被工具化。

每看一个实现文件，都优先找同名测试。例如 `agent-loop.ts` 对应 `agent-loop.test.ts`；不要先试图理解 coding agent 的巨型 interactive mode。

18.4 一个实际定位例子

问题：“为什么两个 tool 同时执行时，界面先显示第二个完成，但历史仍是第一个在前？”

1. 这是 Agent runtime 时间顺序问题，不是 TUI 问题，先到 `packages/agent`；
2. 搜索 `tool_execution_end`，找到 `agent-loop.test.ts` 的 `completion/source order` 测试；
3. 读测试的 `setup` 和断言，确认两种顺序的预期；
4. 再读 `executeToolCallsParallel()`，看“`completion event eager`”与“`persisted result source order`”的实现位置；
5. 最后才看 TUI 如何展示 `end event`。

这个过程说明：好的源码导航从**不**变量开始，不从文件名或 UI 现象开始。

18.5 命令工具箱

当前实现中某符号的定义/使用

```
rg "executeToolCallsParallel" packages/agent
```

某文件的演进史

```
git log --follow -- packages/agent/src/agent-loop.ts
```

某个旧版本文件，不切换工作树

```
git show <commit>:packages/agent/src/agent-loop.ts
```

当前文件和一个 commit 前后的差异

```
git diff <commit>^ <commit> -- packages/agent/src/agent-loop.ts
```

搜索结果只是假设；测试和调用链才是证据。

第 19 章

Mini Pi 实验室：用复刻证明你真的理解

19.1 实验室的角色

mini Pi 不是竞争项目，也不应复制 upstream 的实现。它是**理解校验器**：每次你读懂 Pi 一项机制，就在几十到几百行教学代码中重建它的关键不变量。若自己写不出，或写出的行为无法由测试固定，说明需要回到 Pi 的测试和类型。

19.2 建议的最小数据模型

```
type UserMessage = { role: "user"; content: TextPart[]; timestamp: number };
type AssistantMessage = {
  role: "assistant";
  content: Array<TextPart | ToolCallPart>;
  stopReason: "stop" | "toolUse" | "error" | "aborted";
};
type ToolResultMessage = {
  role: "toolResult";
  toolCallId: string;
  toolName: string;
  content: TextPart[];
  isError: boolean;
};
```

先只支持 text 和 tool call。不要一开始支持图片、thinking、多个 Provider 或 extension。每增加一种 content block，都要重新问：如何流式更新？如何持久化？如何转换为 Provider 负载？如何在 UI 中显示？

19.3 阶段一：scripted model

定义一个不联网的 model：

```
interface Model {
  stream(
    messages: AgentMessage[],
    tools: AgentTool[],
    signal?: AbortSignal
```

```

): AsyncIterable<AssistantDelta>;
}

```

第一条 scripted response 返回 “hello”；第二个版本先返回 `read` tool call，检查输入里是否已有 tool result，再返回总结。这样你可以证明 loop 确实发起了第二次模型调用，而不是应用代码偷偷拼接结果。

19.4 阶段二：工具和错误

给 tool 设置统一 `execute` 契约：

```

type AgentTool = {
  name: string;
  execute(args: unknown, signal?: AbortSignal):
    Promise<{ content: TextPart[] }>;
};

```

未知工具、坏参数、读不到文件都要产生 `isError=true` result。不要在 tool 内把异常吞成空字符串。为每个错误写一个测试：模型能否看到 error result？Agent 是否仍能继续第二轮？

19.5 阶段三：并发与稳定历史

实现两个 `read` tool call 并行。故意让第一个慢、第二个快。你的实现应允许第二个先发“完成” UI event，但 transcript 中 tool result 仍按 assistant 源顺序排列。若只用 `Promise.all` 然后立即按 `resolve` 顺序 `append`，你就复现不了 Pi 的不变量。

19.6 阶段四：session 与 compaction

以 JSONL `append message`；重启后读回并重建 context。然后加入一个非常简单的“超过 N 条消息就产生 summary”规则。你不需要先让模型生成精美摘要；关键是 summary 必须显式进入历史，且测试能说明哪几条原消息被替代。

19.7 mini Pi 的验收表

功能	最小验收
text stream	partial 不写入最终 transcript
tool call	tool result ID 与 call ID 一致
tool error	错误不使整个 Agent 无提示崩溃
parallel tools	completion 与 persistence 顺序分别可断言
abort	final stop reason 是 aborted，后续工具不再启动
session	reload 后下一次模型看到相同上下文
summary	压缩是可见 entry，不是悄悄删除

第 20 章

Provider adapter 解剖：把不同模型协议收敛为一个流

20.1 为什么 adapter 是必要复杂度

OpenAI、Anthropic、Google、Bedrock 等服务的请求字段、tool 格式、thinking、SSE event、cache control 和 usage 名称各不相同。如果 Agent loop 直接面向它们，循环会被大量 provider 条件污染，新增模型也会改变核心行为。Pi 将这种复杂度局限在 `packages/ai/src/api/`。

20.2 adapter 的输入与输出

一个 adapter 的输入包含 Pi 的 `Model`、`Context`、标准 `stream options`；输出是 `AssistantMessageEventStream`。它要完成四类转换：

1. Pi `system/messages/tools` → Provider request；
2. Provider stream events → Pi `text/thinking/tool-call delta`；
3. Provider final usage/stop reason → Pi final assistant message；
4. Pi `session/cache/auth options` → Provider-specific headers/body fields。

20.3 先读 faux，再读一个真实 adapter

`providers/faux.ts` 比真实 adapter 更适合先读，因为它完整展示 Pi 侧事件、usage 和 context 的目标形状，没有 SDK 噪声。理解 faux 后，只选择一个真实 adapter：

- 选择 `api/openai-responses.ts`，学习 Responses API、prompt cache key；
- 或选择 `api/anthropic-messages.ts`，学习 content blocks、thinking、cache control。

不要同时读两者。先写一张“相同 Pi 消息如何映射”的对照表，再扩展到另一个。

20.4 认证不应泄漏到 Agent loop

`Models` 负责把 `credential store`、环境变量、OAuth token 或 Provider-specific auth 解析为请求选项。这样 Agent 只关心“可否 stream”，而不是“token 是否过期”。这一边界也使 status UI 能准确显示“未配置”、“需要重新登录”与“Provider 网络失败”的区别。

20.5 Adapter 阅读问题

1. 此 Provider 如何编码 tool definition 和 tool result?
 2. 哪些 Provider event 被合并为同一种 Pi delta?
 3. provider stop reason 如何变成 Pi 的 stop reason?
 4. usage 的 cache read/write 如何归一?
 5. session ID 在请求中是 header、body key, 还是不使用?
 6. 出错时 stream 是否仍能给 Agent 一个 final result?
- 若这些问题能回答, 你读的不是 SDK 调用, 而是在理解 Pi 的跨 Provider 契约。

第 21 章

终端 UI 深读前需要懂的渲染机制

21.1 为什么终端 UI 仍是一个独立系统

LLM 每秒可生成多次 delta。若每个 delta 都清屏并重画整个聊天记录，终端会闪烁、慢、破坏 scrollbar；复杂 terminal 还会出现光标错乱。因此 `packages/tui` 不是几段 ANSI escape 的集合，而是组件树、终端抽象、输入 buffer、键绑定与增量渲染系统。

21.2 增量渲染的抽象

Agent event

- > component state changes
- > component tree renders terminal lines
- > compare with previous frame
- > emit minimal ANSI updates

这和浏览器 DOM diff 类似，但约束不同：终端宽度会变化、Unicode 宽度不恒定、scrollback 不能乱改、输入编辑器需要光标位置、某些 terminal 不支持所有 capability。

21.3 Pi 的两层 UI

- `packages/tui/src/`: 通用组件，如 text、markdown、editor、select list、terminal；
- `packages/coding-agent/src/modes/interactive/`: Pi 产品组件，如 assistant message、tool execution、footer、model/session selector。

先学通用 TUI 的 render contract，再读 interactive mode 把 `AgentSessionEvent` 转成什么组件。倒过来读会淹没在 import 和产品细节中。

21.4 输入的正确方向

输入并不是直接修改 transcript：

```
keyboard -> editor -> command/input handler
          -> AgentSession.prompt / steer / abort
          -> AgentSession event -> rendered component
```

这保证同一 `AgentSession` 可被 print/RPC 使用，且 session restore 后 UI 只是重放事实，不是重新推断聊天内容。

21.5 第一次 TUI 作业

不要启动完整 interactive mode 后盲目阅读。先找一个简单 `Text` 或 `Markdown` component，理解它如何输出 lines；再找 TUI 如何启动/触发 render；最后追一个 `message_update` 在 interactive mode 中如何更新 assistant component。完成这一小条链之后，再研究 editor、overlay、keybinding。

第 22 章

术语表、检查清单与下一次阅读的人口

22.1 核心术语表

术语	在 Pi 中的准确含义
Agent harness	为模型调用、工具、状态、事件和上下文提供运行时边界的基础设施；不是某个特定 prompt。
turn	一次模型响应及紧随其后的工具批次。一个用户任务可包含多个 turn。
transcript	Agent/session 记录的消息历史。它比单纯 LLM context 更宽，可含 custom/summary entry。
context	发送给模型的 system prompt、转换后的 messages 和 tools。它从 transcript 构建，不必与 transcript 一致。
partial message	还在 stream 中的 assistant 状态，用于 UI；不能当作最终持久化事实。
final message	stream 完成后经 <code>result()</code> 得到的完整 assistant 消息，含最终 usage、stop reason、工具参数。
tool call	assistant content block 中的结构化调用意图，包含 name、arguments 和唯一 call ID。
tool result	工具执行后的标准消息，必须关联 call ID，并显式标记是否 error。
preflight execution mode	工具真正执行前的查找、参数校验、policy/hook 检查阶段。一个 tool batch 的串行或并行策略；副作用工具可要求整个 batch 顺序执行。
steering	正在运行时输入的优先新指令；当前工具批次完成后进入下一 turn。
follow-up	Agent 原本结束后才处理的排队工作。
AbortSignal	取消 stream/tool 的协作信号；不是普通 error。
faux provider	Pi 的确定性假模型，用于不花钱、不联网的 Agent 测试。
Provider	拥有模型列表、认证语义和实际流实现的供应商适配单元。
Models	Provider 集合与认证/委托门面；Agent 通过它间接请求模型。
session ID	本地 session、Provider cache key 或 session affinity 的关联标识，具体映射由 adapter 决定。

prompt cache	服务端复用稳定 prompt 前缀的机制；与本地 JSONL 和 GPU KV cache 不同。
KV cache	推理服务内部保存 attention key/value tensor 的机制；Pi 不直接管理。
compaction	将旧上下文压缩为显式 summary，以满足 context budget 的过程。

22.2 第一次完整调用链的空白模板

每次阅读一个功能，都复制下面模板，不要只写“看了哪些文件”：

用户场景：
 触发入口 (CLI / RPC / UI)：
 模式选择：
 AgentSession 如何创建：
 Agent API：
 传入模型的 Context：
 Provider/stream 事件：
 Agent event：
 工具调用和结果：
 session 写入点：
 UI/print 输出点：
 失败与取消路径：
 相关测试：
 一个关键历史 commit：

22.3 阅读 Agent loop 前的自测

若以下任一问题回答不清，不要急着深入 agent-loop.ts 的 helper：

1. user message 何时进入 agent.state.messages?
2. message_update 中消息为什么不能直接写 session?
3. assistant 返回 tool call 后，为什么还要再调用模型?
4. tool result 的 call ID 为什么不能省略?
5. tool throw 和 Provider stream error 分别变成什么?
6. abort、steering、follow-up 各自在哪个时间点生效?
7. 并发 tool 的完成顺序为何不等于 transcript 顺序?

22.4 阅读 Provider 前的自测

1. 为什么 Agent loop 不应该 import OpenAI 或 Anthropic SDK?
2. Provider 和 Models 的职责如何区分?
3. 为什么 faux provider 对核心测试比真实 API 更可靠?
4. prompt cache、session ID、session JSONL 和 GPU KV cache 的边界是什么?
5. 新增 Provider 时，哪个层应该改变，哪个层不应该改变?

22.5 阅读 Coding Agent 前的自测

- 1. 为什么 AgentSession 不能简单等同 Agent?
- 2. 为什么 cwd 改变要重建 services?
- 3. read/write/bash 分别有哪些不属于 Agent core 的产品风险?
- 4. 为什么 project resource 加载要经过 trust?
- 5. 为什么 interactive UI 不应修改第二份对话历史?

22.6 建议的学习记录评分法

每完成一个纵向切片，自评 0-2 分：

能力	分数	说明
可以复现一个测试/场景	0-2	0= 未运行；1= 运行但不懂；2= 能解释 setup/断言
可以画调用链	0-2	0= 只有文件名；1= 大致路径；2= 有状态和输出边界
可以说明不变量	0-2	0= 只复述代码；1= 知道结果；2= 知道为何不能破坏
可以预测失败路径	0-2	0= 不清楚；1= 知道一种 error；2= 区分 error/abort/retry
可以写最小实验	0-2	0= 无；1= 改了但无测试；2= 有确定性验证
可以解释历史动机	0-2	0= 未查；1= 看过 diff；2= 能说前后不变量变化

总分低于 8 分时，不要进入下一大主题；回到测试或 mini Pi。总分 10 分以上，才值得加入一个新层，例如 session、cache 或 TUI。

22.7 下一次打开源码时，从这里开始

第一条完整学习任务只需这些文件：

```
packages/agent/test/e2e.test.ts
packages/agent/src/agent.ts
packages/agent/src/agent-loop.ts
packages/agent/src/types.ts
packages/ai/src/providers/faux.ts
```

先运行本书第 2 章命令，再从测试中找到 “executes tools and tracks pending tool calls”。不要额外打开 interactive mode、真实 Provider 或 compaction。等你能独立画出它的四消息闭环，再移动到 packages/coding-agent/src/core/agent-session.ts。

第 23 章

第一天的 30 分钟任务

23.1 目标

今天不要理解 Pi，只要建立第一个不依赖 API key 的完整观察。结束时你应该能明确说出：faux model 在哪里设定了两轮响应，工具何时执行，结果如何返回模型。

23.2 时间盒

1. **第 0–5 分钟：**打开 `packages/agent/test/e2e.test.ts`，只找到测试名 `executes tools and tracks pending tool calls`。先不读实现。
2. **第 5–10 分钟：**运行本书第 2 章的 Vitest 命令。确认只有该测试执行；记录 “1 passed”。
3. **第 10–15 分钟：**读该测试中 `faux.setResponses([...])` 的两条 `response`。第一条包含什么 `content block`？第二条为什么是最终文本？
4. **第 15–20 分钟：**读 `packages/agent/test/utis/calculate.ts`，确认 `tool` 接受什么参数、如何生成结果。
5. **第 20–25 分钟：**在 `packages/agent/src/agent.ts` 找到 `prompt()`，在 `agent-loop.ts` 找到 `executeToolCalls()`。只看函数入口与调用关系。
6. **第 25–30 分钟：**用下面模板写五行笔记。

用户消息：

第一次 `assistant message`：

工具调用（名称、`call ID`、参数）：

`tool result`：

第二次 `assistant message`：

23.3 今天不要做什么

不要配置真实 Provider；不要打开 `interactive-mode.ts`；不要尝试理解 `compaction`；不要改生产代码。第一次成功的标准不是修改了什么，而是你能在纸上画出消息闭环，并知道哪条测试能证明它。

下一版会补什么

v1.0 会在本书结构上增加：Agent loop 的逐段源码阅读、AgentSession services/runtime、JSONL/fork/compaction 数据模型、read/edit/write/bash 的实现细节、一个真实 Provider adapter、prompt cache 与 session affinity、extension lifecycle、TUI 增量渲染，以及每个主题四个关键 commit 的演进说明。核心版已经建立这些深读的地图；下一版不会推翻本书，而是沿每章继续向下钻。